

5 套模拟卷及参考答案

按正式考试题型编写：基础简答题 3 道 + 前沿简答题 3 道 + 编程改错题 1 道 + 编程理解题 1 道，每套共 8 题。

模拟卷一

一、基础简答题

1. 请简述人工智能的三次发展浪潮及其代表性技术，并说明当前处于哪一阶段。

参考答案：

- 第一次浪潮（20 世纪 50~60 年代）：以符号主义和逻辑推理为核心，代表性技术为专家系统、知识表示与推理。
- 第二次浪潮（20 世纪 80~90 年代）：以统计学习为核心，代表性技术为支持向量机、决策树、朴素贝叶斯等机器学习方法。
- 第三次浪潮（21 世纪 10 年代至今）：以深度学习为核心，代表性技术为 CNN、RNN、Transformer、大语言模型等。
- 当前处于第三次浪潮的深化阶段，大语言模型、多模态模型与智能体正成为主流方向。

2. 请比较监督学习、无监督学习与强化学习的主要区别，并各举一个典型算法。

参考答案：

- **监督学习**：使用带有标签的数据训练模型，目标是学习输入到输出的映射。典型算法：线性回归、支持向量机、卷积神经网络。
- **无监督学习**：使用无标签数据，目标是发现数据内在结构或分布。典型算法：K 均值聚类、主成分分析、t-SNE。
- **强化学习**：智能体在环境中通过试错获得奖励信号，优化长期累积奖励。典型算法：Q-learning、策略梯度、PPO。

3. 请解释卷积神经网络（CNN）中的“局部连接”“权值共享”和“池化”三个机制，并说明它们分别解决了什么问题。

参考答案：

- **局部连接**：每个神经元只与输入图像的局部区域连接，减少参数数量，同时捕捉局部特征（如边缘、纹理）。
- **权值共享**：同一卷积核在整个图像上滑动共享参数，大幅降低参数数量，使模型具有平移不变性。
- **池化**：对特征图进行下采样（如最大池化），降低特征图尺寸，减少计算量，同时增强对微小形变的鲁棒性。

二、前沿简答题

4. 什么是大语言模型的“对齐”（Alignment）问题？常用的对齐方法有哪些？

参考答案：

- **对齐问题**：大语言模型在预训练阶段学习的是语言概率分布，可能与人类偏好、价值观、安全要求不一致，因此需要进一步对齐。
- **常用方法**：
 - **SFT（监督微调）**：用高质量人工标注数据进行微调。
 - **RLHF（人类反馈强化学习）**：训练奖励模型评估输出质量，再用 PPO 等强化学习算法优化策略。
 - **DPO（直接偏好优化）**：直接用偏好数据优化模型，无需训练单独的奖励模型。
- 对齐后的模型更符合人类意图，减少有害、偏见和幻觉输出。

5. 请解释具身智能（Embodied AI）的核心思想，并说明它与传统“离身”人工智能的主要区别。

参考答案：

- **核心思想**：智能不仅来自抽象计算，还来自智能体与物理环境的交互；通过感知、决策和执行形成闭环。
- **主要区别**：
 - **传统离身 AI**：主要处理符号、文本或图像数据，不直接作用于物理世界。
 - **具身智能**：强调身体（传感器与执行器）和环境的作用，智能体需要在真实或仿真环境中完成任务。
- **典型应用**：机器人导航、抓取、自动驾驶、人形机器人。

6. 请分析生成式人工智能可能带来的两项伦理风险，并说明可采取的治理措施。

参考答案：

- **风险一：虚假内容与深度伪造（Deepfake）**。生成模型可生成逼真的文字、图片、视频，可能被用于造谣、诈骗、色情内容。
- **风险二：版权与数据隐私**。生成内容可能复制训练数据中的受版权保护内容；训练数据可能包含个人隐私信息。
- **治理措施：**
 - 建立内容溯源与水印机制。
 - 对训练数据进行合规审查与去标识化处理。
 - 制定相关法律法规，明确生成式 AI 的责任主体。
 - 开展价值对齐与安全评估。

三、编程改错题

7. 下面代码使用 PyTorch 训练一个二分类模型，请找出 2 处错误并修正。

```
model = Net()
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
criterion = nn.CrossEntropyLoss()

for epoch in range(10):
    for x, y in train_loader:
        outputs = model(x)
        loss = criterion(outputs, y)
        loss.backward()
        optimizer.step()
```

参考答案：

- **错误 1：**每轮迭代没有调用 `optimizer.zero_grad()`。上一轮的梯度会累积，导致训练不稳定。
- **错误 2：**二分类任务使用 `CrossEntropyLoss` 时，模型输出应为未归一化的 logits；若网络最后一层是 `nn.Sigmoid()`，则应使用 `nn.BCELoss`。更常见的修正：保持输出为 logits，使用 `CrossEntropyLoss`，标签 `y` 为长整型类别索引。

修正后代码：

```
model = Net()
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
criterion = nn.CrossEntropyLoss()

for epoch in range(10):
    model.train() # 训练模式
    for x, y in train_loader:
        optimizer.zero_grad() # 清零梯度
        outputs = model(x)
        loss = criterion(outputs, y)
        loss.backward()
        optimizer.step()
```

四、编程理解题

8. 请补全以下 CNN 代码中的 (1)~(4) 空, 使网络能够将 $1 \times 28 \times 28$ 的灰度图像分类为 10 类。

```
class Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d((1), 32, kernel_size=3, padding=1) # (1)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(32, (2), kernel_size=3, padding=1) # (2)
        self.fc1 = nn.Linear((3), 128) # (3)
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu((4))) # (4)
        x = torch.flatten(x, 1)
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return x
```

参考答案:

- (1) 1 : 输入为单通道灰度图。
- (2) 64 : 第二层卷积输出通道数。
- (3) $64 * 7 * 7 = 3136$: 两次池化后特征图为 $64 \times 7 \times 7$ 。

- (4) `self.conv2(x)`。

完整 forward:

```
def forward(self, x):
    x = self.pool(F.relu(self.conv1(x)))      # 1×28×28 → 32×28×28 → 32×14×14
    x = self.pool(F.relu(self.conv2(x)))      # 32×14×14 → 64×14×14 → 64×7×7
    x = torch.flatten(x, 1)                   # 64×7×7 = 3136
    x = F.relu(self.fc1(x))
    x = self.fc2(x)
    return x
```

模拟卷二

一、基础简答题

1. 请解释图灵测试的基本过程，并说明它为何不能作为判断机器真正“理解”语义的标准。

参考答案:

- **基本过程**: 人类询问者通过文本通道分别与人类和机器对话; 若询问者无法稳定区分哪个是机器, 则机器被认为通过测试。
- **不能作为理解标准的原因**:
 - 图灵测试只关注外在行为相似性, 不考察内部机制。
 - 机器可能通过模式匹配、记忆大量语料或语言技巧“伪装”理解, 而非真正拥有语义、常识或意识。
 - 它无法评估感知、推理、创造、情感等多元智能维度。

2. 什么是过拟合和欠拟合? 分别列举两种缓解方法。

参考答案:

- **欠拟合**: 模型过于简单, 在训练集和测试集上表现都差。缓解: 增加模型复杂度、增加特征、训练更久。
- **过拟合**: 模型过度记忆训练数据, 训练集表现好但测试集差。缓解:
 - 增加训练数据。
 - 正则化 (L1/L2)。
 - Dropout。

- 早停 (Early Stopping) 。
- 交叉验证。

3. 请说明循环神经网络 (RNN) 处理序列数据的基本原理, 以及 LSTM 为何能缓解 RNN 的梯度消失问题。

参考答案:

- **RNN 基本原理:** 在每个时间步接收当前输入和上一时刻隐藏状态, 计算当前隐藏状态并输出; 参数在时间步间共享。
- **梯度消失问题:** 长序列反向传播时, 梯度连乘导致指数级缩小, 模型难以学习长期依赖。
- **LSTM 缓解方法:** 引入门控机制 (输入门、遗忘门、输出门) 和细胞状态, 使信息可以在细胞状态中相对不变地传递, 减少梯度消失, 增强长期记忆能力。

二、前沿简答题

4. 请解释大语言模型中“上下文学习” (In-context Learning) 的含义, 并说明它与传统微调的主要区别。

参考答案:

- **上下文学习:** 在提示中给出几个示例 (few-shot), 模型无需更新参数即可根据示例推断任务模式并完成新任务。
- **与传统微调的区别:**
 - 上下文学习不修改模型权重, 仅通过提示改变行为。
 - 微调需要额外的训练数据和反向传播更新参数。
- 上下文学习依赖模型在预训练阶段学习到的泛化能力和模式匹配能力。

5. 多模态大模型在训练时面临哪些主要技术挑战? 请列举三项并简要说明。

参考答案:

- **模态对齐:** 文本、图像、音频的语义空间不同, 需要跨模态对齐方法 (如对比学习) 建立统一表示。
- **数据获取与标注:** 高质量配对的多模态数据稀缺, 标注成本高。
- **计算开销:** 多模态模型参数量和训练数据量巨大, 对算力和存储要求极高。
- **幻觉与跨模态推理错误:** 模型可能在不同模态间产生不一致或错误推断。

6. 什么是 AI Agent? 请说明它与大语言模型的关系, 并分析其潜在风险。

参考答案:

- **AI Agent**: 能够感知环境、设定目标、制定计划、调用工具并执行多步骤任务的智能系统。
- **与大语言模型关系**: 大语言模型可作为 Agent 的“大脑”, 负责理解意图、推理规划和生成指令; Agent 通过工具调用将模型能力延伸到现实世界。
- **潜在风险**:
 - 工具调用错误导致实际损失 (如错误交易、错误预订)。
 - 权限过大可能泄露隐私或越权操作。
 - 目标设定不当可能产生不可控行为。

三、编程改错题

7. 以下代码在测试集上评估准确率, 但结果每次运行都不稳定, 请找出问题并修正。

```
model = Net()
model.train() # 重点检查

correct = 0
total = 0
with torch.no_grad():
    for x, y in test_loader:
        outputs = model(x)
        _, predicted = torch.max(outputs, 1)
        total += y.size(0)
        correct += (predicted == y).sum().item()
accuracy = correct / total
```

参考答案:

- **错误**: 测试前没有调用 `model.eval()`, 模型仍处于训练模式。如果模型包含 Dropout 或 BatchNorm, 会导致每次预测结果随机变化, 且 BatchNorm 使用当前批次统计量而非全局统计量。
- **修正**: 在测试循环前添加 `model.eval()`。

```
model.eval() # 切换到评估模式
correct = 0
total = 0
with torch.no_grad():
    for x, y in test_loader:
        outputs = model(x)
        _, predicted = torch.max(outputs, 1)
        total += y.size(0)
        correct += (predicted == y).sum().item()
accuracy = correct / total
```

四、编程理解题

8. 某 RNN 模型用于文本分类，输入为长度 20 的词向量序列，每个词向量维度为 128。RNN 隐藏层维度为 64，输出为 5 类。请补全网络结构。

```
class TextRNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.rnn = nn.RNN(input_size=(1), hidden_size=(2), batch_first=True)
        self.fc = nn.Linear((3), (4))

    def forward(self, x):
        out, _ = self.rnn(x) # out shape: (batch, 20, 64)
        out = out[:, -1, :] # 取最后时刻输出
        out = self.fc(out)
        return out
```

参考答案：

- (1) 128：输入词向量维度。
- (2) 64：RNN 隐藏层维度。
- (3) 64：最后时刻输出维度。
- (4) 5：分类类别数。

模拟卷三

一、基础简答题

1. 请简述人工智能的技术范畴，并说明“感知、学习、推理、决策、生成”五项能力各自的含义。

参考答案：

- **技术范畴：**包括机器学习、深度学习、自然语言处理、计算机视觉、语音识别、机器人学、知识图谱、强化学习等。
- **五项能力：**
 - 感知：从环境获取信息，如图像识别、语音识别。
 - 学习：从数据中提取规律并改进性能。
 - 推理：根据已有知识进行逻辑推断和因果分析。
 - 决策：在多个选项中选择最优行动。
 - 生成：创造新的内容，如文本、图像、音乐、代码。

2. 请解释 Transformer 中的“自注意力机制”（Self-Attention）为何能建立序列中不同位置之间的联系，并说明其与 RNN 的主要区别。

参考答案：

- **自注意力机制：**为序列中每个位置生成 Query、Key、Value，通过点积计算任意两个位置之间的相似度（注意力权重），再加权求和得到新的表示；因此每个位置都能直接关注序列中其他所有位置。
- **与 RNN 的区别：**
 - RNN 依赖时间步逐步传递信息，难以捕捉长距离依赖，且难以并行计算。
 - Transformer 可直接建模全局依赖，计算可高度并行化，是现代大语言模型的基础架构。

3. 什么是 Dropout？它在训练阶段和测试阶段的行为有何不同？为什么测试阶段需要关闭 Dropout？

参考答案：

- **Dropout：**一种随机正则化技术，训练时以一定概率随机将部分神经元输出置零，防止神经元共同适应训练数据。
- **行为差异：**
 - 训练阶段：随机失活神经元，每个 batch 的网络结构不同。
 - 测试阶段：所有神经元参与计算，否则输出会因随机性不稳定，导致同一输入多次预测结果不同。
- **关闭原因：**测试需要确定性输出，用于稳定评估和部署。

二、前沿简答题

4. 请解释“检索增强生成”（RAG）的基本原理，并说明它如何缓解大语言模型的幻觉问题。

参考答案：

- **RAG 原理：**在生成回答前，先从外部知识库中检索与问题相关的文档或片段，将检索结果作为上下文输入模型，再基于这些事实生成回答。
- **缓解幻觉：**
 - 模型生成内容依赖于检索到的真实资料，而不是完全依赖内部参数记忆。
 - 即使模型参数中缺乏某些知识，也能通过检索补充最新、准确的信息。
 - 便于追溯答案来源，提高可信度。

5. 请分析多模态 AI 相比单一模态 AI 的优势，并以自动驾驶场景为例说明不同模态的作用。

参考答案：

- **优势：**多模态 AI 能同时利用多种信息源，提高感知和理解能力，增强鲁棒性和场景适应能力。
- **自动驾驶例子：**
 - 摄像头图像：识别车道线、交通标志、行人、车辆。
 - 激光雷达点云：精确测距和三维建模。
 - 毫米波雷达：在恶劣天气下检测障碍物速度和距离。
 - GPS/地图/文本指令：提供全局路径规划和语义信息。
- 多模态融合使系统决策更全面、更安全。

6. 请说明“价值对齐”在 AI 安全中的重要性，并列举两种实现价值对齐的技术路径。

参考答案：

- **价值对齐：**确保 AI 系统的目标、行为与人类价值观和意图一致，避免产生有害、偏见或不可控的结果。
- **重要性：**随着 AI 能力增强，未对齐的系统可能以违背人类利益的方式优化目标，造成伦理和安全风险。
- **技术路径：**
 - **RLHF：**利用人类反馈训练奖励模型，引导模型输出符合人类偏好。
 - **Constitutional AI / 红队测试：**通过原则约束和对抗性测试发现潜在风险并修正。

三、编程改错题

7. 以下代码在 GPU 上训练，但运行时出现张量不在同一设备的错误，请找出问题并修正。

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = Net().to(device)

for x, y in train_loader:
    x = x.to(device)
    outputs = model(x)
    loss = criterion(outputs, y)
    loss.backward()
    optimizer.step()
```

参考答案：

- **错误：**输入 `x` 被移动到 GPU，但标签 `y` 仍在 CPU，导致 `criterion(outputs, y)` 时设备不一致。
- **修正：**将 `y` 也移动到同一设备。

```
for x, y in train_loader:
    x = x.to(device)
    y = y.to(device) # 关键修正
    optimizer.zero_grad()
    outputs = model(x)
    loss = criterion(outputs, y)
    loss.backward()
    optimizer.step()
```

四、编程理解题

8. 请补全以下代码，使 Transformer 编码器第一层的输出维度正确。

```
class TransformerEncoderLayer(nn.Module):
    def __init__(self, d_model=512, nhead=8):
        super().__init__()
        self.self_attn = nn.MultiheadAttention((1), (2), batch_first=True)
        self.fc = nn.Linear(d_model, d_model)

    def forward(self, x):
        attn_output, _ = self.self_attn(x, x, x)
        x = x + attn_output
        x = self.fc(x) + (3)
        return x
```

参考答案:

- (1) `d_model` : `MultiheadAttention` 的 `embed_dim`。
- (2) `nhead` : 注意力头数。
- (3) `x` (残差连接) : `self.fc(x) + x` 。

完整代码:

```
class TransformerEncoderLayer(nn.Module):
    def __init__(self, d_model=512, nhead=8):
        super().__init__()
        self.self_attn = nn.MultiheadAttention(d_model, nhead, batch_first=True)
        self.fc = nn.Linear(d_model, d_model)

    def forward(self, x):
        attn_output, _ = self.self_attn(x, x, x)
        x = x + attn_output          # 残差连接
        x = self.fc(x) + x          # 残差连接
        return x
```

模拟卷四

一、基础简答题

1. 请简述感知机 (Perceptron) 的结构与原理, 并说明单层感知机无法解决 XOR 问题的原因。

参考答案:

- **结构与原理**：感知机接收多个输入，计算加权求和并通过阶跃函数输出；本质是用一个超平面将数据分为两类。
- **无法解决 XOR**：
 - XOR 的四个样本在二维平面上无法用一条直线线性可分。
 - 单层感知机只能解决线性可分问题，无法表示非线性决策边界。
- **解决方法**：引入多层感知机（MLP）和非线性激活函数。

2. 请解释“梯度下降法”的基本思想，并说明学习率过大或过小分别会导致什么问题。

参考答案：

- **基本思想**：通过计算损失函数对参数的梯度，沿梯度反方向更新参数，使损失逐步减小。
- **学习率过大**：更新步长过大，可能跳过最优点，导致震荡或发散。
- **学习率过小**：更新步长过小，收敛速度极慢，容易陷入局部最优或需要过多迭代。

3. 请说明强化学习中“状态、动作、奖励、策略、价值函数”五个核心概念的含义。

参考答案：

- **状态 (State)**：智能体在环境中某一时刻的观测或内部表示。
- **动作 (Action)**：智能体在当前状态下可执行的操作。
- **奖励 (Reward)**：环境对智能体动作给出的即时反馈信号。
- **策略 (Policy)**：从状态到动作的映射，决定智能体如何选择动作。
- **价值函数 (Value Function)**：评估从某个状态或状态-动作对出发，未来能获得的累积奖励期望。

二、前沿简答题

4. 请解释“提示工程”（Prompt Engineering）在大语言模型应用中的作用，并列举三种常用技巧。

参考答案：

- **作用**：通过设计和优化输入提示，引导大语言模型生成更符合预期、更准确、更安全的输出，而无需修改模型参数。
- **常用技巧**：
 - **角色扮演**：让模型以特定角色回答（如“你是一位资深教师”）。
 - **Few-shot 示例**：在提示中给出几个输入-输出示例。
 - **链式思维 (Chain-of-Thought)**：要求模型“一步一步思考”，提高复杂推理任务的准确率。

5. 请比较 GPT 与 BERT 在模型结构、训练目标和典型应用上的差异。

参考答案：

维度	GPT	BERT
结构	Decoder-only（自回归）	Encoder-only（双向）
训练目标	预测下一个词	掩码语言模型 + 下一句预测
生成能力	强，适合文本生成	弱，主要用于理解
典型应用	文本续写、对话、代码生成	文本分类、NER、语义相似度

6. 请分析人工智能系统可能产生的偏见（Bias）来源，并提出两项缓解措施。

参考答案：

- 偏见来源：
 - 训练数据分布不均衡，某些群体样本不足。
 - 历史数据中本身就存在社会偏见。
 - 特征选择或标注过程引入主观偏见。
- 缓解措施：
 - 数据层面：增加少数群体样本，进行数据增强和重采样。
 - 算法层面：使用公平性约束、对抗性去偏等方法。
 - 评估层面：使用公平性指标（如不同群体准确率差异）进行测试。

三、编程改错题

7. 以下代码实现了 Dropout + BatchNorm 的网络训练，测试时发现准确率异常低。请找出问题。

```
class Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(784, 256)
        self.bn = nn.BatchNorm1d(256)
        self.dropout = nn.Dropout(0.5)
        self.fc2 = nn.Linear(256, 10)

    def forward(self, x):
        x = F.relu(self.bn(self.dropout(self.fc1(x))))
        x = self.fc2(x)
        return x
```

参考答案:

- **错误:** **Dropout** 放在了 **BatchNorm** 之前。训练时随机丢弃部分神经元，会导致进入 BatchNorm 的样本分布剧烈变化，影响统计量估计；测试时 BatchNorm 使用训练阶段的全局统计量，而训练统计量已被 Dropout 干扰，导致性能下降。
- **修正:** 通常顺序为 **Linear** → **BatchNorm** → **ReLU** → **Dropout**。

```
def forward(self, x):
    x = self.dropout(F.relu(self.bn(self.fc1(x))))
    x = self.fc2(x)
    return x
```

四、编程理解题

8. 请补全以下代码，使网络能正确处理 **3×64×64** 的输入图像并输出 5 类。

```
class Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 16, kernel_size=5, padding=2)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(16, 32, kernel_size=5, padding=2)
        self.fc1 = nn.Linear((1), 128)
        self.fc2 = nn.Linear(128, (2))

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))      # 16×32×32
        x = self.pool(F.relu(self.conv2(x)))      # 32×16×16
        x = torch.flatten(x, 1)                   # (3)
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return x
```

参考答案：

- (1) $32 * 16 * 16 = 8192$ ：展平后维度。
- (2) 5：输出类别数。
- (3) 1：`torch.flatten(x, 1)` 的维度参数。

模拟卷五

一、基础简答题

1. 请说明人工智能的“能力边界”主要体现在哪些方面，并举例说明。

参考答案：

- **常识与因果推理**：模型可能识别相关关系，但缺乏真正的因果理解和物理常识。
- **可解释性**：深度模型决策过程复杂，难以解释。
- **数据依赖**：性能高度依赖训练数据分布，泛化到分布外场景困难。
- **价值判断**：AI 缺乏人类的伦理判断和情感理解。
- **举例**：自动驾驶在罕见极端天气下可能失效；语言模型可能生成语法正确但事实错误的回答。

2. 请解释搜索算法中“无信息搜索”与“有信息搜索”的区别，并各举一例。

参考答案：

- **无信息搜索：**不了解目标位置的启发信息，仅按固定策略遍历状态空间。例如 BFS（广度优先）、DFS（深度优先）、Dijkstra。
- **有信息搜索：**利用启发函数估计到目标的代价，优先扩展更有希望的节点。例如 A* 算法、贪心最佳优先搜索。
- **区别：**有信息搜索通常效率更高，但启发函数设计不当可能导致次优解或不完整。

3. 请说明卷积神经网络中“感受野”的含义，并解释为何深层网络具有更大的感受野。**参考答案：**

- **感受野：**卷积神经网络中某个特征图单元对应的输入图像区域大小。
- **深层网络感受野更大：**
 - 每一层卷积核都在前一层特征图上操作，相当于将多个小卷积核的感受野组合起来。
 - 池化操作进一步扩大了后续神经元能覆盖的原始输入区域。
 - 因此深层特征能捕捉更大范围的上下文和高级语义。

二、前沿简答题

4. 请解释“链式思维提示”（Chain-of-Thought Prompting）为何能提升大语言模型在数学推理任务上的表现。**参考答案：**

- **原理：**通过提示让模型显式生成中间推理步骤，而不是直接输出答案。
- **提升原因：**
 - 将复杂问题分解为多个简单子问题，降低单步推理难度。
 - 中间步骤提供了可解释的推理路径，减少跳步导致的错误。
 - 使模型能够利用预训练阶段学习到的程序性知识和符号推理能力。

5. 请说明视觉 Transformer（ViT）与传统 CNN 在图像处理上的主要差异。**参考答案：**

- **输入处理：**
 - CNN：通过局部卷积核逐层提取特征，强调局部性和平移不变性。
 - ViT：将图像切分为若干补丁（patch），将每个 patch 当作 token 输入 Transformer。
- **全局建模：**

- CNN 依赖堆叠层扩大感受野来捕捉全局信息。
- ViT 通过自注意力机制直接建模任意两个 patch 之间的关系。
- **数据需求**：ViT 通常需要更大规模的数据和计算资源进行预训练。

6. 请分析 AI 智能体在调用外部工具时可能面临的安全风险，并提出两项防护措施。

参考答案：

- **风险**：
 - 工具调用错误或被恶意构造的输入利用，导致数据泄露、未授权操作或资金损失。
 - Agent 可能基于错误反馈反复执行有害操作。
 - 权限过大导致越权访问敏感资源。
- **防护措施**：
 - **权限最小化**：只授予 Agent 完成任务所需的最小权限。
 - **输入校验与沙箱**：对用户输入和工具参数进行严格校验，工具执行在隔离环境中进行。
 - **人工确认机制**：高风险操作前要求人类审批。

三、编程改错题

7. 以下代码用于保存模型，但加载后发现预测结果与训练时不一致。请找出可能原因并修正。

```
# 训练
model.train()
for epoch in range(10):
    train_one_epoch(model, train_loader)

torch.save(model, 'model.pth')

# 测试
model = torch.load('model.pth')
model.train() # 重点检查
with torch.no_grad():
    for x, y in test_loader:
        outputs = model(x)
```

参考答案：

- **问题**：测试时调用了 `model.train()`，使 Dropout 和 BatchNorm 处于训练模式，导致预测结果不稳定或错误。
- **修正**：测试前应调用 `model.eval()`。

```
model = torch.load('model.pth')
model.eval() # 关键修正
with torch.no_grad():
    for x, y in test_loader:
        outputs = model(x)
```

四、编程理解题

8. 请补全以下用于图像分类的残差块（ResNet BasicBlock），使输入输出维度一致。

```
class BasicBlock(nn.Module):
    def __init__(self, in_channels):
        super().__init__()
        self.conv1 = nn.Conv2d(in_channels, in_channels, kernel_size=3, padding=1)
        self.bn1 = nn.BatchNorm2d(in_channels)
        self.conv2 = nn.Conv2d(in_channels, in_channels, kernel_size=3, padding=1)
        self.bn2 = nn.BatchNorm2d(in_channels)

    def forward(self, x):
        identity = (1)
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        out = F.relu(out + (2))
        return out
```

参考答案：

- (1) `x`：保存输入作为残差连接的恒等映射。
- (2) `identity`：将输入加到卷积输出上，形成残差连接。

完整 forward：

```
def forward(self, x):
    identity = x
    out = F.relu(self.bn1(self.conv1(x)))
    out = self.bn2(self.conv2(out))
    out = F.relu(out + identity)
    return out
```

附录：答题时间分配建议

题型	题数	建议总时长	每题时长
基础简答题	3	30 min	10 min/题
前沿简答题	3	45 min	15 min/题
编程改错题	1	20 min	20 min/题
编程理解题	1	25 min	25 min/题
合计	8	120 min	—

提示：简答题按“定义 → 要点 → 举例/影响”三段式作答，控制在 200~300 字；编程题先写状态切换，再写参数计算，注意 `model.eval()`、`torch.no_grad()`、`optimizer.zero_grad()` 等关键点。